# SC2002 Group Assignment Report

Restaurant Rush

**Lab Group:** MACS1, Group 4


**Group Members:**

1. LIM ZHITONG (U2521509L)
2. LEM SHI JYE (U2520502E)
3. AKSHITH MATTAPARTHI (U2521557H)
4. JAIN SAAMYA (U2540868D)
5. ZANDER HEE HAO YANG (U2522074E)


**Submission Date:** 13 November 2026

# 1. UML Class Diagrams

- **Stage 1 UML Class Diagram:** Details baseline classes (e.g. core entities, simple order processing, basic ingredient inventory, and main menu UI).
- **Stage 2 UML Class Diagram:** Shows extensions to Stage 1 (e.g., customer seating strategies, kitchen order queues, special event modifiers).
- **Stage 3 UML Class Diagram:** Shows full system architecture including persistent game saves/state tracking, advanced order priority strategies, and dynamic ingredient deterioration.

# 2. UML Sequence Diagram

- **Stage:** Stage 3
- **Starting Conditions:** A VIPCustomer instance with ID 3 and arrival turn 7 is initialized. The test uses the Stage 3 menu order (Salad, Burger, Pasta, Salad + Burger Combo, Salad + Pasta Combo). A VIPCustomer loses 5 satisfaction per turn while waiting for a table or food, versus a regular customer's 8 and gets 10% off the menu price.
- **Outcome:** The VIP customer chooses the correct item based on the customer ID selection algorithm, calculates the bill with a 10% VIP discount (paying 1620 cents = $16.20 instead of 1800 cents), enters WAITING status, and loses 5 satisfaction points (reducing satisfaction from 100 to 95).
- **Object Interaction & Logic:**
    1. Object Initialization & Metadata Checks: The test case instantiates VIP: VIPCustomer(3, 7). It verifies the type (getType() -> "VIP") and checks base satisfaction loss per waiting turn (getSatisfactionLoss() -> 5).
    2. Item Selection Logic: chooseItem(menu) queries menu.size() (5) and uses the inherited customer formula menu.getItem((id - 1) % menu.size()), corresponding to index 2 (Pasta).
    3. Bill Calculation & Discount Application: calculateBill(Pasta) queries Pasta: MenuItem for getPriceCents() (1800 cents). The VIPCustomer then invokes Money.percentOf(1800, 90) to apply a 10% discount, yielding 1620 cents ($16.20).
    4. Satisfaction & Waiting State Tracking: The initial satisfaction is 100. Upon calling loseSatisfactionForWaiting(), the object invokes an internal self-call getSatisfactionLoss() (5 points lost) and updates satisfaction to 95 while in WAITING status.

# 3. Design Considerations and Use of OO Concepts

## 3.1 Key Abstractions

| Abstraction | Responsibility in Restaurant Rush | OO Concept / Principle | Example in the project |
|---|---|---|---|
| Menu item (abstract class) | MenuItem defines shared name, price, and preparation-time operations. Dish and ComboMeal provide item-specific behavior. | Abstraction, inheritance, polymorphism, composition | ComboMeal contains two menu items and derives its price and preparation units from them. |
| Customer(abstract class) | Holds shared customer data and service lifecycle. | Abstraction, inheritance, polymorphism | Regular, VIP and critic customers share the same lifecycle. |
| Customer subclasses | Define customer-specific waiting, ordering and billing rules. | Method overriding, polymorphism | Critics choose the most expensive item while VIPs get a discount. |
| Order and order status | Track order details, preparation and progress. | Encapsulation, state representation | The chef advances an order until it is ready. |

| | | | |
|---|---|---|---|
| Restaurant | Manages customers, tables, queues and orders. | Encapsulation, Single Responsibility Principle | Staff use it to seat customers and place orders. |
| Staff and role subclasses | Share turn rules while implementing role-specific tasks. | Abstraction, inheritance, polymorphism | The host, waiter, chef and cashier perform different jobs. |
| Game controller and console interface | The controller coordinates game rules while the interface handles input and display. | Separation of concerns | The interface collects choices while the controller applies game rules. |

## 3.2 SOLID Principles Demonstrated

| Principle | Application |
|---|---|
| Single Responsibility Principle (SRP) | Classes have focused roles: the chef prepares orders, the scoreboard records results, and the console interface handles interaction. |
| Open/Closed Principle (OCP) | New customer or menu-item types can extend existing abstractions with limited changes to their callers. |
| Liskov Substitution Principle (LSP) | Subclasses work wherever their parent type is expected, such as a dish being used as a menu item. |

| | |
|---|---|
| Interface Segregation Principle (ISP) | Focused abstractions, such as the customer factory, represent specific responsibilities. |
| Dependency Inversion Principle (DIP) | Abstractions such as menu items are used at some boundaries while the controller still coordinates concrete components. |

## 3.3 Extensibility and Maintainability

| Design Area | Benefit | Example |
|---|---|---|
| Customer behavior | Customer rules stay in their respective subclasses. | A critic chooses the most expensive item while a VIP calculates a discounted bill. |
| Menu items | Different items share one common type. | Dishes and combo meals can both be handled as menu items. |
| Customer lifecycle | An enum clearly represents service stages. | Stages include waiting, seated and eating. This is not the State pattern because each state is not a separate behavior class. |

## 3.4 Trade-offs and Alternatives Considered

| Design Decision | Alternative Considered | Why the Alternative Was Less Suitable | Trade-off |
|---|---|---|---|
| Abstract customer class | A customer interface implemented independently by each type. | It would not share customer data or lifecycle logic, leading to duplication or extra shared classes. | The abstract class reuses common state and behavior, but uses Java's single inheritance slot. |
| Abstract menu-item class | A menu-item interface. | The items share a name and display behavior as well as method signatures while an interface alone could duplicate that code. | The abstract class centralizes shared behavior, but also uses the single inheritance slot. |
| Enum for customer progress | Separate State-pattern classes. | The current lifecycle is small and fixed, so separate classes would add unnecessary complexity. | The enum is simple while more complex state behavior may eventually benefit from separate classes. |

# 4. Reflections

## Lim Zhitong:

- **What Did You Learn?**
  I learned how object-oriented design principles such as abstraction, encapsulation and polymorphism help make a software system easier to understand and extend. I also learned that making good design decisions early can provide a foundation for carrying a project forward as its requirements become more complex. Use-case and sequence diagrams were valuable for understanding what users can do and how the system's components interact.

- **How Did You Apply It?**
  In Restaurant Rush, we used shared abstractions such as Customer and MenuItem, with subclasses for different customer types and menu items. This lets us represent different behaviors while keeping common logic in one place. I also used use-case diagrams to clarify the game's functionality and sequence diagrams to follow interactions between the console interface, game controller and other parts of the system.

- **What Challenged You?**
  It was challenging to connect the high-level design to the implementation and understand how a change in one part of the game could affect other components. As the game flow involves customers, staff, orders and turn-based rules, it was important to keep each class's responsibilities clear.

- **How Did You Solve It?**
  I used the diagrams to break down the system's requirements and interactions before considering how they should be implemented. Relating those flows to the project's classes helped me understand where responsibilities belonged and how the components worked together. This reinforced the value of clear design: it makes the current system easier to follow and gives future extensions a stronger starting point.

## Lem Shi Jye:

- **What did you learn?**
  We learned how to use diagrams such as UML class and sequence diagrams as well as good design principles to craft out a game for our Restaurant Rush Group Assignment. We used Class Diagrams to structure the game and sequence diagrams to model the behaviours of each customer over the course of the game. This allows us to effectively structure and carry out our project.

- **How did you apply it?**
  We applied it through a platform called github, where we can collectively do the project without the need of sharing one device. This allows us to do our project efficiently without waiting for others to finish their part. This is due to the fact that we can now do remotely using multiple accounts instead of from a single account. Additionally, github also is a convenient place for us to backtrack on our progress, and fill in any gaps that need to be filled in.

- **What challenged you?**
  The hardest part is navigating the UI in GitHub as I personally have not used it before. There are so many functions that we need to use such as branches and tags to complete the code. Due to my lack of experience, it took some time to learn the relevant functions and tools. This caused a reduction in efficiency and negated some of the benefits of using github such as boosting efficiency. However, I believe that knocking how to use github now would benefit me in the long run as after this project, I would gain more knowledge about github to use it efficiently on future projects

- **How did you solve it?**
  I solved the problem in a stage manner. I went to google the solution first but the solutions are sometimes inaccurate. Next, I relied on artificial intelligence (AI) to directly tell me the solution but the AI can sometimes also be inaccurate. As a last resort, I relied on peers and collective research through various mediums like youtube and reddit forums to accurately identify the solutions to navigate around GitHub such as branching.

## Akshith Mattaparthi:

- **What did you learn?**
  The most useful thing I learnt was how to use Git and GitHub properly. Before this project I only saw GitHub as a place to upload files. Now I understand that `git add`, `git commit` and `git push` are three separate steps, that branches let each member work without disturbing `main`, and that tags such as `stage-1`, `stage-2` and `stage-3` mark the exact commit where each stage is finished and runnable.

- **How did you apply it?**
  Our game is built in three stages, and each stage extends the one before it. Tags let us keep a runnable snapshot of every stage, so we can compare Stage 2 with Stage 3 and see exactly what changed. This matches an idea from class: extend working code instead of rewriting it. In my Stage 3 work, `ComboMeal` extends `MenuItem`, so `Order` and `Chef` treat a combo like any other dish with no changes. I used the same thinking for version control: I'll commit Stage 3 as small steps (`HappyHour`, `ComboMeal`, then `Host`) on top of the saved Stage 2 code. I also made the Stage 3 class diagram and [checked it against the code].

- **What challenged you?**
  The hardest part was using GitHub, and working with many Java files at once added to it. Our game has around 30 classes spread across five packages (boundary, control, entity, customer and staff). At first I couldn't tell which class did what, or which file to change when I wanted to add a feature. For my Stage 3 work I had to follow how `GameController`, `Waiter`, `Host`, `HappyHour` and `ComboMeal` all connect, and one small change could affect several files. On GitHub, branches, tags and the commands around them were confusing too, and our first repo ended up with a messy history, with all the stage folders together, no tags and very few branches. It was hard because a wrong git command isn't always easy to undo, and I couldn't always see what it had done

- **How did you solve it?**
  For the Java files, the class diagram helped the most. Drawing it showed me how the classes relate and which package each one belongs to. I also learnt to read the code one stage at a time, comparing Stage 2 with Stage 3 to see which files changed and why, instead of trying to understand everything at once. For GitHub, I watched videos on how it works and used AI to break each idea into small parts, and I made sure I understood each explanation before running a command. Our group then started a new repo and is rebuilding the history in stage order, so every tag points to a real finishing commit. What I learnt is that it's better to understand a tool step by step than to rush it, and that AI is most useful when I check what it says and test it myself.

## Jain Saamya:

- **What did you learn?**
  One of the main things I learned from this project was how object-oriented programming concepts work together in a larger application, rather than just in individual Java exercises. I gained a better understanding of abstraction, inheritance, encapsulation and polymorphism, particularly through the Customer and MenuItem classes. I also learned how important it is to maintain consistent interfaces between classes when different team members are working on separate parts of the same project. Beyond Java, I became more comfortable using Git and GitHub for collaborative development.

- **How did you apply it?**
  My main contribution was working on the Stage 1 entity and customer model classes, including Customer, RegularCustomer, MenuItem, Dish, Order and Table. I helped organize and integrate these classes into our project, ensuring they compiled successfully and used interfaces consistent with the gameplay components developed by my teammate. For example, the abstract MenuItem class defines common methods such as getPriceCents() and getPrepUnits(), while Dish provides their implementation. This allows other classes to interact with menu items through a common abstraction. I

also used a separate Git branch to organize my contribution before pushing it to the shared repository.

- **What challenged you?**
  One of the biggest challenges was understanding how the different classes depended on one another and how they would eventually fit into the complete game. Since our group divided the project into different components, I had to understand not only the classes I was responsible for, but also how they interacted with the staff and game controller classes. Keeping the package structure and method interfaces consistent was important to ensure smooth integration. I also found GitHub challenging initially, particularly understanding branches, commits and how to manage my work without affecting other team members. Additionally, I encountered file-permission issues on my Mac while setting up the project.

- **How did you solve it?**
  I approached these challenges by breaking the project into smaller parts and understanding the responsibilities of each class individually. I studied the relationships between the entity and customer classes and compiled them together to verify that their dependencies were correctly handled. For the GitHub workflow, I learned to use commands such as git add, git commit and git push step by step, allowing me to maintain a separate feature branch and contribute without interfering with the main codebase. I also used terminal commands to diagnose the file-permission issues and resolve them. Overall, this project taught me that successful software development involves not only understanding programming concepts, but also managing dependencies, testing code and collaborating effectively with a team.

## Zander Hee Hao Yang:

- **What did you learn?**
  I learned about Java OOP programming and that its modular features like inheritance and abstraction allowed for code designs that were reusable, simple and flexible. The capability to create multiple classes and have them interact with each other via methods allowed my team to split work efficiently while still building towards a central framework.

- **How did you apply it?**
  I applied OOP concepts when creating the central game framework for Stage 1. For instance, the various staff objects were bundled together under an abstract Staff class, each inheriting basic attributes from the Staff class while adding additional unique attributes and methods of their own.

- **What challenged you?**
  Learning to use GitHub posed a significant challenge to me as I was unfamiliar with its features and design. It was rather different from other productivity applications I was familiar with, which made using it unintuitive at first. I understood that it was an online

repository for programmers to store and share their codes, but did not understand the purpose of creating branches or tagging releases. I had to spend time investigating how to maneuver the user interface and learn about its features.

- **How did you solve it?**
  I looked up tutorials and guides on how to use GitHub and spent time testing different features on my own. I also sought help from my team mates when I was lost and studied the project documentation in detail.

# 5. GitHub Repository & Evidence Links

- **GitHub Repository:** https://github.com/ZanderFish/SC2002-MACS1-Group-4
- **Stage Runnable Tags:** stage-1, stage-2, stage-3

# 6. Declaration of Original Work

We hereby declare that the attached group assignment has been researched, undertaken, completed, and submitted as a collective effort by the group members listed below.
We have honored the principles of academic integrity and have upheld Student Code of Academic Conduct in the completion of this work. We understand that if plagiarism is found, lower or no marks will be awarded and disciplinary actions may be taken.